

# Assignment 2: LLM Chatbots and Vector Databases

Rajat Murhe | B.Tech Computer Science (AI and Machine Learning), VIT Chennai

## 1. Self assessment

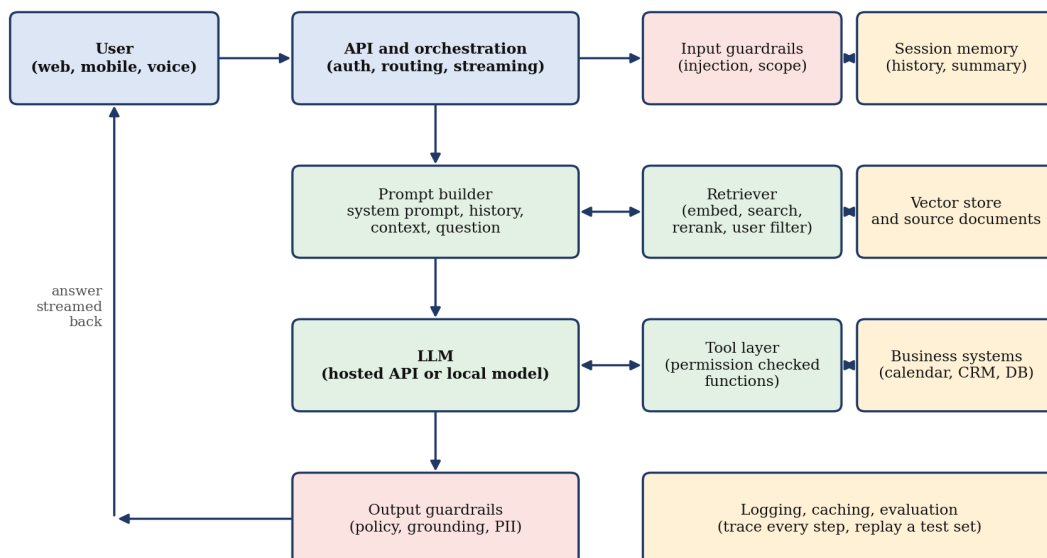
Scale: A means I can code it independently, B means I can code it under supervision, C means I have little or no understanding.

| Area          | Rating | What the rating is based on                                                                                                                                                                                                                          |
|---------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LLM           | A      | Built a multi agent voice assistant with tool calling and a safety layer (ClinicVoice AI), and RAG systems on a vector database (RAG Intelligence, AI Job Assistant). Worked with Gemini and locally hosted models through Ollama.                   |
| Deep Learning | A      | Fine tuned a BERT model in PyTorch and benchmarked it against Random Forest and XGBoost (NeuroAI). Designed my own PyTorch detection architecture and training pipeline (RV Vision 0.1). This rating is for applied work, not research level theory. |
| AI            | A      | Agentic workflows with tool use and human in the loop review (FieldProof AI), and reinforcement learning agents trained with PPO (TradeAlpha, indoor navigation).                                                                                    |
| ML            | A      | Scikit learn and XGBoost models with cross validation and hyperparameter search (CreditX), plus SHAP explanations (NeuroAI).                                                                                                                         |

## 2. Key architectural components of an LLM based chatbot

**Approach in one paragraph.** The model is only one part of a chatbot. Most of the quality, safety and cost of the final product comes from the system around it. A request arrives, is checked, is enriched with the right context (conversation history plus relevant facts retrieved from the company's own data), is sent to the model, may trigger tools that act on real systems, and the answer is checked again before the user sees it. Figure 1 shows the components and how a request moves through them.

Figure 1. Components of an LLM chatbot and how a request flows through them



The two columns on the right are supporting systems. The left column is the main path. Red boxes are the safety checks, amber boxes are stores and external systems.

## The components

| Component                       | What it does                                                                                           | Typical choices                                                          | What goes wrong without it                                                                     |
|---------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Interface                       | Takes the message, shows the reply as it streams in.                                                   | Web or mobile app, voice front end, Slack or WhatsApp bot                | Users wait for the full answer and feel the latency.                                           |
| API and orchestration           | Authenticates the user, routes the request, runs the steps in order.                                   | FastAPI or Node backend, a graph or state machine for multi step flows   | Logic ends up inside prompts, which cannot be tested or controlled.                            |
| Session memory                  | Keeps the conversation state per user and trims or summarizes old turns to fit the context window.     | Redis or a database, rolling summary                                     | The bot forgets context, or the prompt grows until it is slow and expensive.                   |
| Prompt builder                  | Assembles the system prompt, history, retrieved context and the question into one request.             | Templates with clear sections                                            | The model mixes instructions with data, which also opens the door to prompt injection.         |
| Retrieval (RAG)                 | Finds the passages that answer the question and filters them by what this user may see.                | Embedding model, vector database, optional keyword search and reranker   | The model answers from memory and invents details, or leaks documents the user should not see. |
| LLM                             | Generates the answer or decides which tool to call.                                                    | Hosted API or local model, kept behind an interface so it can be swapped | Provider lock in, and no fallback when a provider is down.                                     |
| Tool layer                      | Runs actions such as booking, lookups or ticket creation.                                              | Defined functions with typed inputs, permission checks, audit logs       | The model could write to systems directly, with no validation or record.                       |
| Guardrails                      | Check the input (injection, off topic requests) and the output (policy, grounding, sensitive data).    | Rules, classifiers, a second model as a judge                            | Unsafe or invented content reaches the user.                                                   |
| Logging, caching and evaluation | Trace every step, reuse answers to repeated questions, and replay a fixed test set after every change. | Tracing tools, semantic cache, a test set with expected answers          | Nobody can tell whether a change made the bot better or worse.                                 |

## How one request flows

- 1 The user sends a message. The backend authenticates them and loads their session.
- 2 Input guardrails check for prompt injection and out of scope requests.
- 3 The retriever embeds the question, searches the vector store with a filter for what this user may access, and returns the top passages.
- 4 The prompt builder combines the system prompt, recent history, the passages and the question.
- 5 The LLM either answers or asks to call a tool. The tool layer checks permissions, runs the function, and returns the result to the model.
- 6 Output guardrails check the draft answer against policy and against the retrieved sources.
- 7 The answer streams back to the user, and every step is logged for later debugging and evaluation.

## Design decisions that matter most

- **RAG before fine tuning.** Company knowledge changes often. Retrieval lets me update a document and have the bot use it immediately, and I can show the source of every answer. Fine tuning is better for changing tone or format, not for storing facts.
- **The model proposes, the backend decides.** The model never touches a database or an external system itself. It asks for a tool call, and the backend validates the request and checks the user's permission before running it.
- **Check both sides.** Input filtering alone is not enough, because some failures start inside the model's own output.
- **Measure, do not guess.** A fixed test set, including adversarial questions, is replayed after every prompt, model or retrieval change.
- **Control latency and cost.** Stream the answer, cache repeated questions, send only the context that is needed, and use a smaller model for simple steps.

**An example from my own work.** In ClinicVoice AI, a voice assistant for a medical clinic, I tested the system by trying to break it. The model invented an identity verification step that was never real clinic policy. An input filter could never have caught that, because the problem came from the model's own output. I fixed it with an output level check, and added a global safety gate before routing so every agent is covered, not only the FAQ path.

## 3. Vector databases

### What a vector database is

An embedding model turns a piece of content, such as a paragraph of text, an image or an audio clip, into a list of numbers called a vector. Content with similar meaning ends up close together in this number space, even if it uses different words. A vector database stores these vectors with their original text and metadata, and answers one main question quickly: *which stored items are closest in meaning to this query?*

Closeness is measured with a distance function. Cosine similarity compares direction and is the usual choice for text. Dot product and Euclidean distance are the other common options.

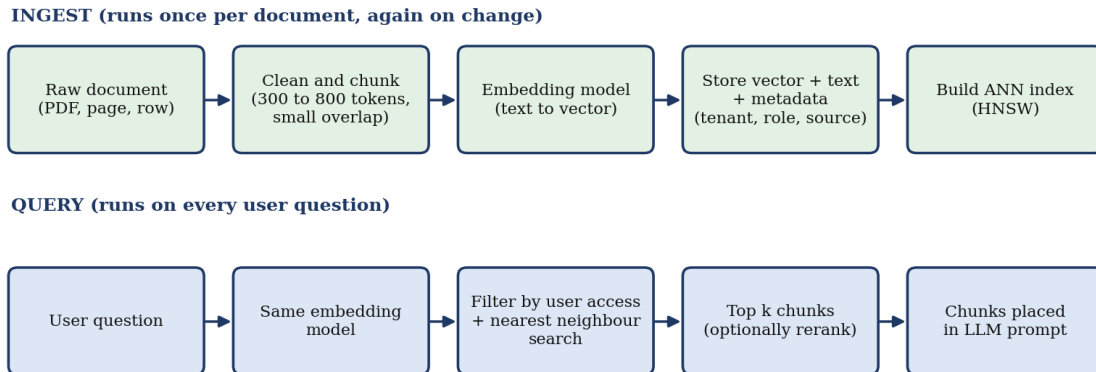
### Why not just compare against everything

Exact search compares the query to every stored vector, so cost grows with the size of the data. At millions of vectors that is too slow for an interactive chatbot. Vector databases use **approximate nearest neighbour (ANN)** indexes. They accept a small, tunable loss in recall to return results in milliseconds.

| Index idea   | How it works                                                                                         | Strength                                                     | Cost                                              |
|--------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------|
| HNSW         | A layered graph. Search starts at a coarse layer and walks toward closer neighbours, layer by layer. | Fast queries and high recall, works well for most workloads. | Uses more memory and builds more slowly.          |
| IVF          | Clusters the vectors, then searches only the few clusters nearest to the query.                      | Lower memory, faster to build.                               | Recall depends on how many clusters are searched. |
| Quantization | Compresses each vector into fewer bytes.                                                             | Cuts memory a lot, which matters at very large scale.        | Some loss of accuracy.                            |

Two more features decide whether a vector database is usable in a real product. **Metadata filtering** restricts a search to rows that match conditions such as tenant, department or document type. **Hybrid search** combines

keyword matching with vector search, which helps with exact terms like product codes or names that embeddings handle poorly.



**Figure 2. Ingest and query pipeline of a vector database backed retrieval system**

Documents are chunked, embedded and indexed once. At question time the same embedding model is applied to the query, and the top chunks go into the LLM prompt.

## Choosing a vector database for a hypothetical problem

**The problem.** An internal assistant for a company that answers employee questions from HR policies, engineering documents, contracts and finance reports.

- About 3 million chunks across 40 departments, growing slowly.
- Different roles may see different documents, so every search must respect access rights, and permission changes must take effect immediately.
- A few queries per second at peak, with answers expected within a few seconds.
- The team already runs PostgreSQL and has two engineers, with no dedicated infrastructure team.

**The candidates, judged against this problem.**

| Option                | What it is                                                      | Strength                                                                                      | Limit for this problem                                                            |
|-----------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| pgvector (PostgreSQL) | A vector extension for Postgres with HNSW and IVF indexes.      | Filters and joins are plain SQL, data stays transactionally consistent, no new system to run. | Reaches a ceiling at very large scale or very high query rates.                   |
| Qdrant                | Open source vector engine with filtering built into the search. | Fast filtered search, quantization, self hosted or managed.                                   | A second system to run, secure and keep in sync with the source of truth.         |
| Pinecone              | Fully managed vector service.                                   | Almost no operations work, scales on demand.                                                  | Ongoing cost, and company documents and permissions leave our own infrastructure. |
| Milvus                | Distributed engine built for very large collections.            | Handles billions of vectors with sharding.                                                    | Heavy to operate for a 3 million chunk problem.                                   |
| FAISS or Chroma       | A search library, and a lightweight embedded store.             | Excellent for notebooks and prototypes.                                                       | No built in access control or production management.                              |

### My choice: pgvector with an HNSW index.

- 1 **Access control is the hardest requirement, and Postgres already solves it.** The permission filter and the similarity search run in one SQL query, joined against the tables that already hold users, roles and document rights.
- 2 **The vectors cannot drift out of sync.** If a document is deleted or its permissions change, the vectors change in the same transaction. With a separate vector database I would need a sync job, and every sync job is a place for a stale or leaked result.
- 3 **The scale fits.** A few million vectors with an HNSW index is comfortably inside what Postgres handles on ordinary hardware.
- 4 **It matches the team.** One system to back up, monitor and secure. I have also used pgvector in my own projects (RAG Intelligence and AI Job Assistant), so it is the option I can build and debug fastest.

A sketch of the core query:

```
CREATE INDEX ON chunks USING hnsu (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

SET hnsu.ef_search = 80;

SELECT id, text, source
FROM chunks
WHERE department_id = ANY(:allowed_departments)
ORDER BY embedding <=> :query_embedding
LIMIT 5;
```

**The honest caveat.** With an approximate index, a very selective filter can make the index return fewer than the requested number of rows, because filtering is applied to the candidates the index finds. I would handle this by partitioning or indexing by department for the largest groups, raising the search breadth parameter, and using the iterative index scans added in recent pgvector versions. I would confirm which of these is needed by testing, not assuming.

### When I would choose something else

| Situation                                                  | Better choice      | Why                                                                                             |
|------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------|
| Hundreds of millions of vectors, or a very high query rate | Qdrant or Milvus   | Purpose built engines with sharding and quantization scale further than a single Postgres node. |
| Small team, spiky traffic, no one to operate a database    | Pinecone           | Managed and serverless, in exchange for cost and giving up control of where data lives.         |
| Heavy need for combined keyword and vector ranking         | Weaviate or Qdrant | Hybrid search is a first class feature.                                                         |
| Offline experiments in a notebook                          | FAISS or Chroma    | Fastest way to test an embedding model or chunking strategy.                                    |

**How I would validate the decision before committing.** I would build a test set of real questions where I know which source passage contains the answer. At the expected data size, I would measure recall and response time with and without the access filter, for the top two candidates. The numbers on this test set would decide the final choice, not the feature list.